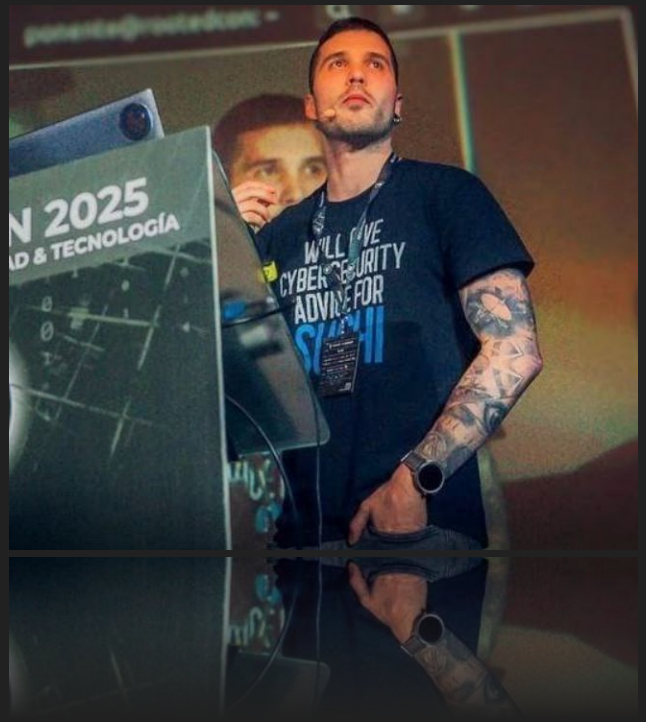
Kill Chain Reloaded: Abusing legacy paths for stealth persistence

Alejandro Hernando @0xedh & Borja Martínez @borjmz

0xedh@defcon33:~$ whoami

Alejandro Hernando / @0xedh



Red team and researcher in the Hacking Accenture Spain team.

I like to break things in my free time and spend money on gadgets.

Github: @0xedh
Telegram: @edhx0
Twitter: @0xedh

accenture

borjmz@defcon33:~$ whoami

Borja Martínez / @borjmz / vorga

Red team and researcher in the Hacking Accenture Spain team.

From time to time I play the occasional CTF and was part of the ID-10-T (Retired) team.

Telegram: @borjmz
Twitter: @Qm9yamFN
Github: @borjmz

accenture

Table of Contents

01

Research of vulnerable Artifacts

Contextualization: Why does it matter to talk about this now?

- UEFI bootkits are no longer theoretical: they are real, active and used by APTs.
- BlackLotus (2023): the first known malware to bypass Secure Boot in Windows 11.
- Pre-SO persistence = total control + EDR evasion.
- Digital signatures and Secure Boot are not enough if vulnerable binaries are not revoked.

Secure Boot ≠ secure by default

- Secure Boot depends on an updated chain of trust.
- Revoked binaries are stored in the DBX (UEFI) database.
- It is easy to load legitimate but vulnerable bootloaders if the DBX is out of date.

The CrowdStrike case (2024)

- Failed update caused massive BSODs on thousands of endpoints.
- Real impact: banks, airports, hospitals, critical services offline.
- A single piece of software in the boot environment = whole system down.
- What if this was done by a malicious bootkit?

Real bootkits in the real world

Here is a quick table with some of the most known bootkits that have been used in real operations, many of them linked to APTs

Bootkit	Year	Persistence / Bypass Vector
LoJax (APT28)	2018	SPI Firmware (UEFI)
MosaicRegressor	2020	Custom BIOS
MoonBounce (APT41)	2021	UEFI SPI implant – fileless in memory
ESPecter	2021	EFI System Partition (ESP)
BlackLotus	2022–23	UEFI bootloader + Secure Boot bypass (CVE-2022)
CosmicStrand	2021–22	SPI Firmware implant – unknown delivery method
CVE-2024-7344	2024–25	Signed .efi – Secure Boot bypass via RT loader
CVE-2025-3052	2025	NVRAM write – disables Secure Boot on boot

Most of them are loaded before the operating system, which means that no EDR, antivirus or traditional solution can see them. They control the system from the moment you turn it on.

The objective

- To find signed software/firmware with useful functions for post-exploitation.
- We are looking for:
  - Drivers with MmMapIoSpace, ZwMapViewOfSection, MmCopyMemory, memcpy, etc.
  - Vulnerable (signed) UEFI drivers -> to load bootkits.
  - Vulnerable native signed windows apps (wpbbin.exe)
- Tools:
  - VirusTotal + RetroHunt -> YARA rules + retrohunting of old drivers.
  - Microsoft Catalog -> downloaded .CAB of legitimate signed drivers.
  - Own scripts -> string extraction, fast static analysis, hash matching, etc.

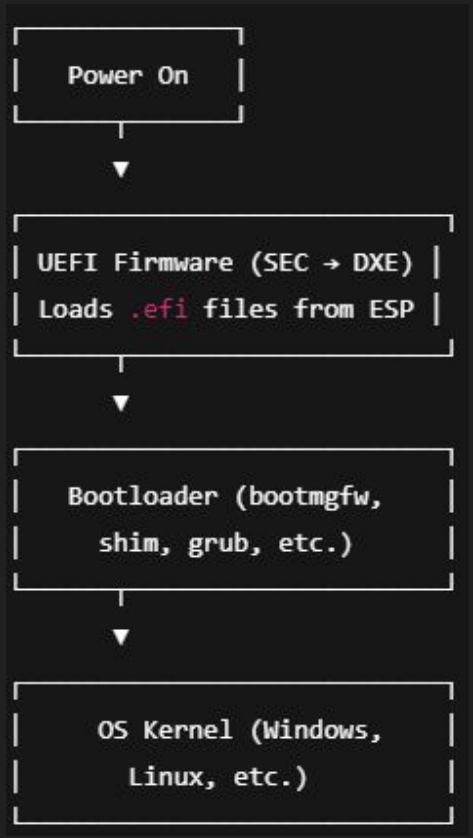

Let’s start the demo
two clicks after
made with mematic

02

UEFI Exploitation

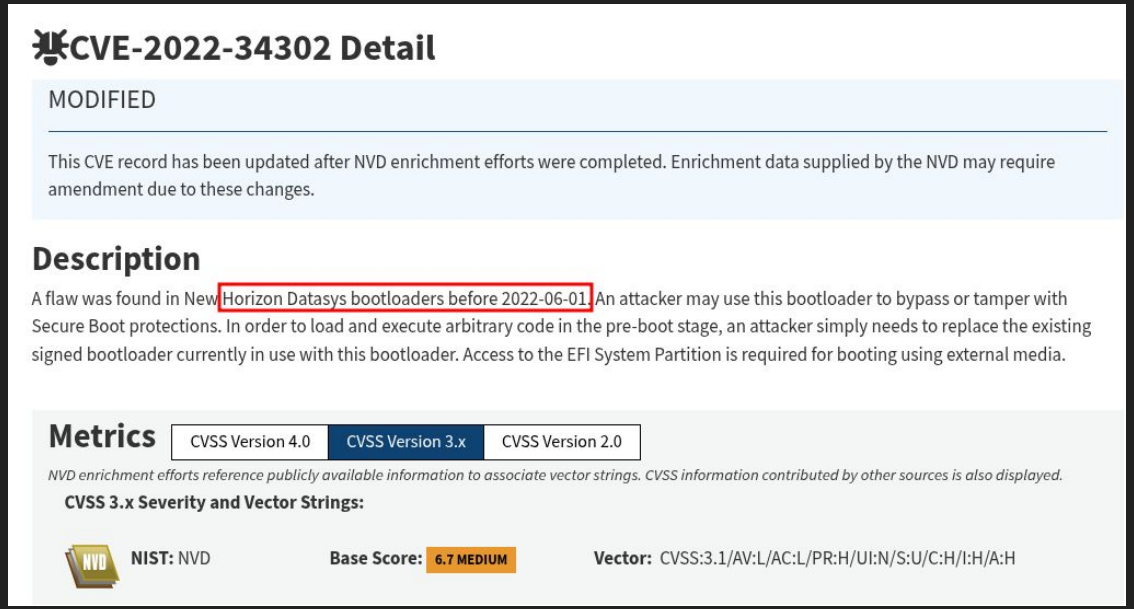
UEFI and Bootloader – The Real Entry Point

- UEFI is the first software that runs after powering on the system.
- Acts like a mini OS: has its own heap, services, and .efi modules.
- Loads components from the EFI System Partition (ESP) – bootloaders and drivers.
- The bootloader (bootmgfw.efi, shim.efi, etc.) launches the OS.
- Everything before ExitBootServices() runs outside OS/EDR visibility.
- If we load our signed .efi at this stage, we get pre-OS full control.
- We can redirect the boot flow, persist, or disable protections.
- This is the foundation for the loading chain shown in the next demos.



Vulnerable bootloaders

- There are a lot of vulnerable applications in the wild, as an example, **CVE-2022-34302**



CVE-2022-34302 Detail

MODIFIED

This CVE record has been updated after NVD enrichment efforts were completed. Enrichment data supplied by the NVD may require amendment due to these changes.

Description

A flaw was found in New Horizon Datasys bootloaders before 2022-06-01. An attacker may use this bootloader to bypass or tamper with Secure Boot protections. In order to load and execute arbitrary code in the pre-boot stage, an attacker simply needs to replace the existing signed bootloader currently in use with this bootloader. Access to the EFI System Partition is required for booting using external media.

Metrics

CVSS Version 4.0 | CVSS Version 3.x | CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

NIST: NVD **Base Score:** 6.7 MEDIUM **Vector:** CVSS:3.1/AV:L/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:H

Microsoft DBX revocation list

- https://github.com/microsoft/secureboot_objects


```
{
  "authenticodeHash": "CBFA2A86144EB21D65A6B17245BAD4F73058436C7292BE56DC6EBAB29DA61606",
  "hashType": "SHA256",
  "flatHash": "A4A5D536E11A12E1023ED51ACDBA64107C5B463822D31C6F5F7855B32FF031D1",
  "filename": "BiosFlashShell-efi64-81.02.efi",
  "description": "CVE-2025-3052",
  "companyName": "DT Research Inc",
  "dateOfAddition": "2025-05-07",
  "signingAuthority": "CN = Microsoft Corporation UEFI CA 2011"
},
{
  "authenticodeHash": "9D7E7174C281C6526B44C632BAA8C3320ADD0C77DC90778CC148938829F45E5E",
  "hashType": "SHA256",
  "flatHash": "1BA53A168EA9F0C2835BBD11EC1DD4DCAE9D8C9312F8FFEE1362359844674870",
  "filename": "Dtbios-efi64-70.17.efi",
  "description": "CVE-2025-3052",
  "companyName": "DT Research Inc",
  "dateOfAddition": "2025-05-07",
  "signingAuthority": "CN = Microsoft Corporation UEFI CA 2011"
},
```

Microsoft DBX revocation list

- **C3D65E174D47D3772CB431EA599BBA76B8670BFAA51081895796432E2EF6461F** = Good old **CVE-2022-34302**
- And the other ones?

```
./RebootRestoreRxPro113_2706604790/Reboot Restore Rx Pro/x64/shdloader.efi
DBX match missing for hash: 3115ab78b7a47a432f4a9747b032458d1f8fde403d20e0cbe9b6a7658b33f83c
./RebootRestoreRx33-20201014/x64/shdloader.efi
DBX match missing for hash: 3115ab78b7a47a432f4a9747b032458d1f8fde403d20e0cbe9b6a7658b33f83c
./RebootRestoreRxProv10.7-20170920/Reboot Restore Rx Pro v10.7 20170920/Reboot Restore Rx Pro/
{
  "authenticodeHash": "C55BE4A2A6AC574A9D46F1E1C54CAC29D29DCD7B9040389E7157BB32C4591C4C",
  "hashType": "SHA256",
  "flatHash": "C3D65E174D47D3772CB431EA599BBA76B8670BFAA51081895796432E2EF6461F",
  "filename": "shdloader.efi",
  "description": "CVE-2022-34302",
  "companyName": "New Horizon Datasys Inc",
  "dateOfAddition": "2022-08-01",
  "signingAuthority": "CN = Microsoft Corporation UEFI CA 2011"
}
```

Reboot Restore Analysis

- Firmware looks for the **ESP**'s \EFI\Boot**bootx64.efi**.
- **Shdloader.efi** is renamed or copied over **bootx64.efi**.
- When the firmware invokes "bootx64", it actually runs **Shdloader.efi** first.
- **Shdloader.efi** then implements its own logic (decompression, signature checks, etc.) before finally loading the real **bootmgfw.efi**.

```
drwxr-xr-x 2 edh edh    4096 may 15 09:15 .
drwxr-xr-x 4 edh edh    4096 may 15 09:15 ..
-rwxr-xr-x 1 edh edh 2774984 may 15 09:25 bootx64.dat
-rwxr-xr-x 1 edh edh  306440 may 15 09:25 bootx64.efi
-rwxr-xr-x 1 edh edh  197376 may 15 09:25 shdmgr.ef_
-rwxr-xr-x 1 edh edh   41408 may 15 09:25 Shield.efi
```

#PHASE 1 - What does “shieldloader” do?

- Opens the ESP’s FAT volume
- Locates the file \EFI\Boot**shdmgr.ef_** (**notice the underscore, proprietary NAUY format**)
- Allocates the file’s size, and reads all bytes into a buffer

```
load_shdmgr_wrapper(
  L"\\EFI\\Boot\\shdmgr.ef_",
  L"--shdmgr",
  1,
  loadedImageHandle
);
```

```
status = filesystem_read(path, &raw_buf, &raw_size);
if (status < 0 || raw_size == 0) {
  // fallback to firmware-volume embedded copy
  for (i = 0;  i < fv_count;  i++) {
    if (!load_raw_ef_vol(raw_buf, i, /*first*/1, flags)) {
      FreePool(raw_buf);
      continue;
    }
    break;
  }
  if (i == fv_count) {
    return (word*)EFI_NOT_FOUND;
  }
```

#PHASE 2 - Decompress the “NAUY” format

- Inspects the **0x200-byte header** at the start
- Reads the **ASCII magic “NAUY”**, and extracts the compressed and uncompressed lengths from the header
- Walks through the chunks, decompress or memcpy

```
ok = parse_NAUY_header(
       raw_buf,
       raw_size,
       &pe_scratch,     // large scratch area >= uncompressed_total_length
       &uncompressed_len
     );
FreePool(raw_buf);
if (!ok) {
  return (word*)EFI_COMPROMISED_DATA;
}

// Allocate final buffer and copy out the full PE image
pe_buf  = AllocatePool(uncompressed_len);
ok = copy_img(pe_scratch, pe_buf, uncompressed_len, &read_len);
```

The “NAUY” format

```
0x0000 - 0x01FF  0x200-byte HEADER

  uint16  header_size         = 0x0200
  uint16  reserved/version    (unused)
  char[4] magic               = 'N' 'A' 'U' 'Y'
  uint32  total_uncompressed_length
  uint32  total_compressed_length
  (padding/reserved to 0x200)
```

```
                    CHUNK RECORDS
Repeat until you've consumed total_compressed_length bytes:

  uint16 record_len          // includes this 4-byte hdr
  uint8  reserved            // = 0
  uint8  flags               // bit2 (0x04) = compressed
  uint8  payload[record_len-4]
    ├─ if (flags & 0x04) -> LZ4_decompress(payload)
    └─ else              -> memcpy(payload)
```

```
                    800-byte TRAILER

  char[4]   magic      = 'R' 'S' 'A' 'S'
  uint32    hash_len   // # of leading bytes signed
  uint32    fmt_ver    // format/version (0x00010000)
  uint32    sig_len    // always 256
  uint8[12] padding
  uint8[516] public-key DER blob (ignored at runtime)
  uint8[256] RSA-PKCS#1-v1.5 signature
```

#PHASE 3 - Crypto signature verification

- It jumps to the **last 800 bytes**, “RSAS” followed by 256-byte RSA signature.
- If the magic is wrong, the padding check fails, or the digest doesn’t match, **check_signature()** returns false, and the loader aborts with **EFI_SECURITY_VIOLATION**.

```
// Phase 3: Cryptographic check on the PE blob
ok = check_signature(pe_buf, uncompressed_len, &DAT_0003D530);
if (!ok) {
  return (word*)EFI_SECURITY_VIOLATION;
}
```

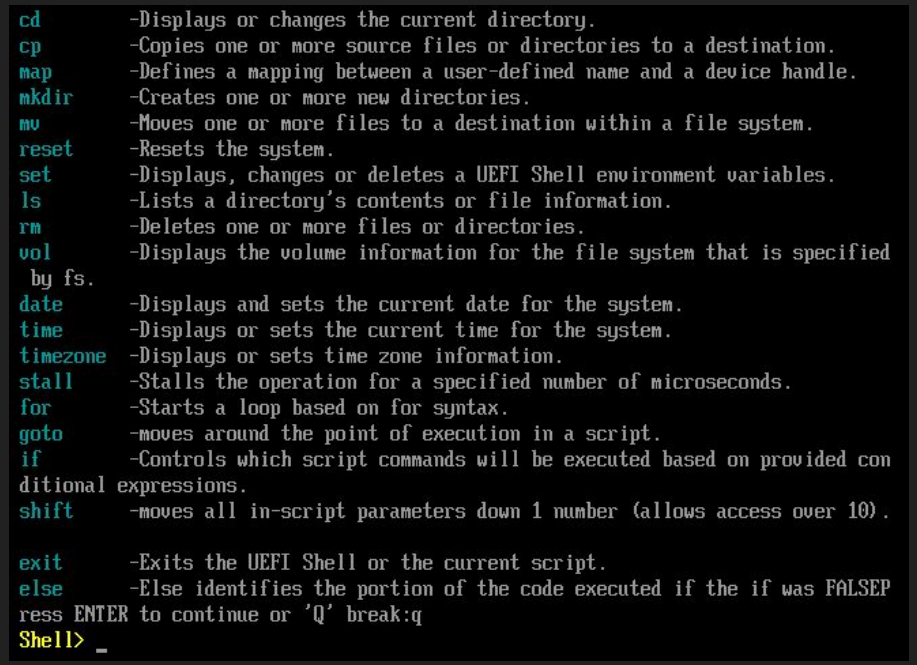
#PHASE 4 - Starting the bootmanager

- Finally, the loader hands off to the standard UEFI Boot Services. It calls **LoadImage()** and returns a new EFI handle. It then invokes **StartImage()**, which transfers control into the loaded PE.

```
  // Phase 5: UEFI load & start
  image_handle = bootservices_loadimage(gBS, pe_buf, uncompressed_len, cmdline);
  thunk_FUN_000020dc();  // install LoadOptions
  return (word*)(*LoadImage_Start)(image_handle, NULL);
}
```

The “shell.ef_” container

- Just a few commands... But better than nothing.



What does “shdmgr.ef_” do?

- After decompressing the EFI in shdmgr.ef_, the loader skips the custom format and directly loads another EFI image.

```
pwVar7 = L"\\EFI\\Boot\\Shield.efi";
load_unsigned_efi(param_1, param_2, pwVar7, flags);
if (DAT_0003889f & 1) {
  print(L"Press any key to continue loading windows (Microsoft Boot Manager)…");
  FUN_00005b04();  // wait for key
}
```

- No signature verification: **load_unsigned_efi()** never calls **check_signature()**.
- No container parsing: it expects a straight PE image, so you can drop any *.efi there.
- It actually contains its own minimal PE loader and relocator.

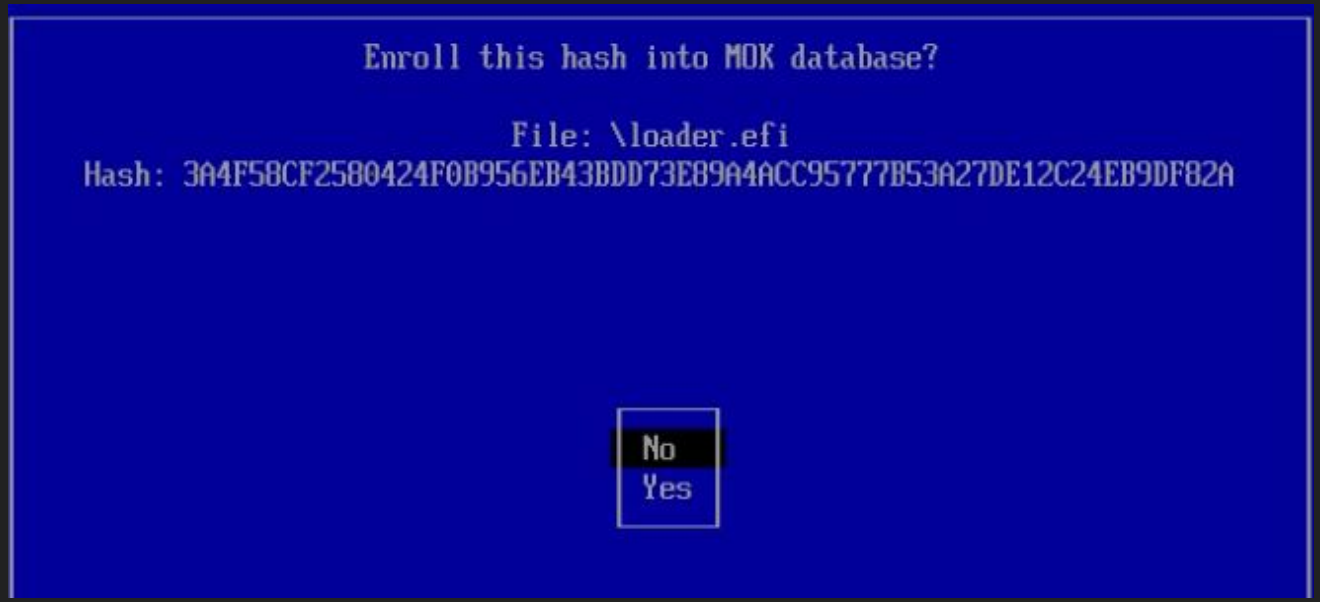

Demo Time !

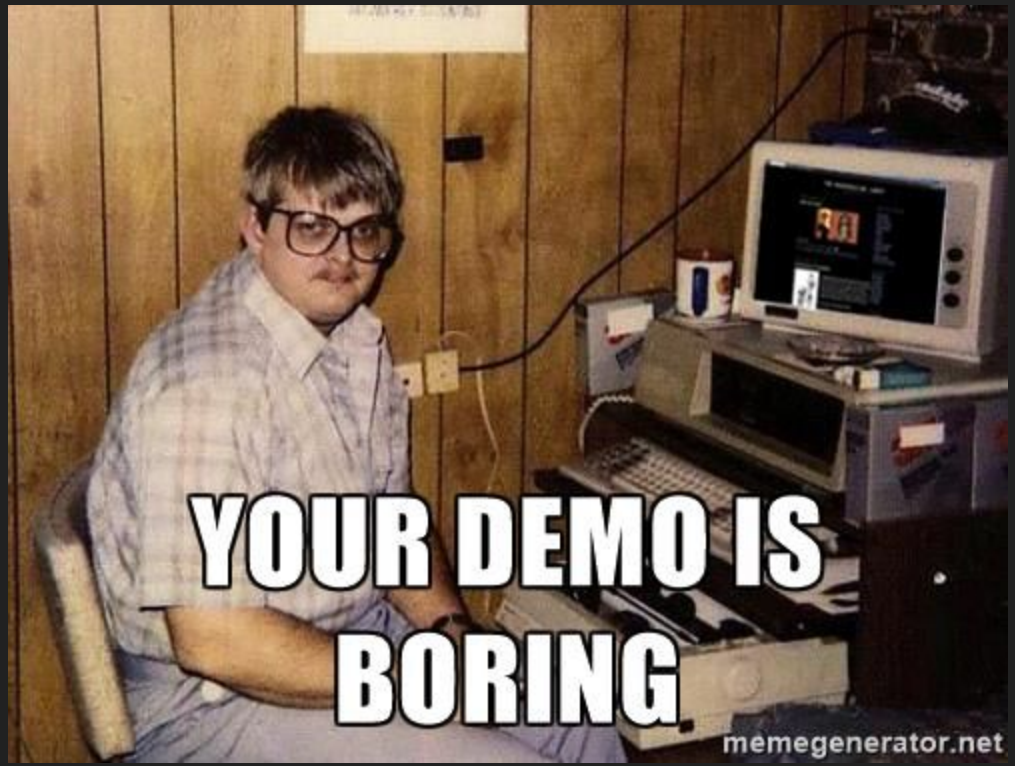
Bitlocker protection

- When you enable BitLocker on an OS volume, the VMK is sealed in the TPM against the current values of several Platform Configuration Registers (PCRs).
- With Secure Boot enabled, PCR 7 records a hash of every EFI executable that runs before the kernel: bootmgfw.efi, BCD, early drivers,etc.
- If any one PCR is different the unseal fails and Windows falls back to BitLocker recovery mode.

Bitlocker protection

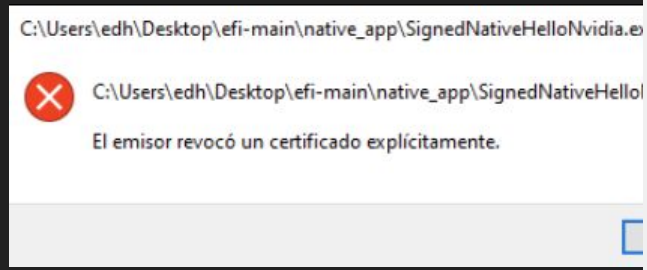
- Use WMI to run DisableKeyProtectors with **DisableCount = 0**
- Deploy and overtwrite boot files, replace **shdmgr.ef_** with **shell.ef_**
- “**stage0.efi**”, a signed tool capable of enrolling unsigned binaries into MOK (knoppix).




YOUR DEMO IS
BORING
memegenerator.net

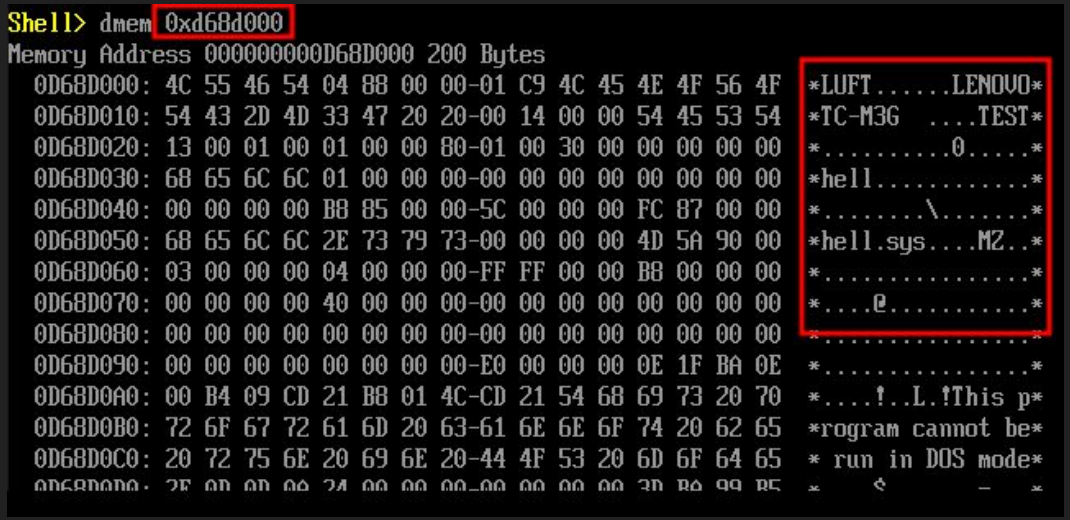
Enough ASCII art! Let's do something

- What about **WPBT** (Windows Platform Binary Table)?
- WPBT lets the firmware place one native app (signed) at C:\Windows\System32 and executes it in the session manager context.
- In updated Windows 11 24H2, you can't just use a leaked code signing certificate.
- Simply dropping an unsigned "malicious.exe" via WPBT is "useless", the file lands on disk but nothing launches it.



Lenovo wpbbin.exe

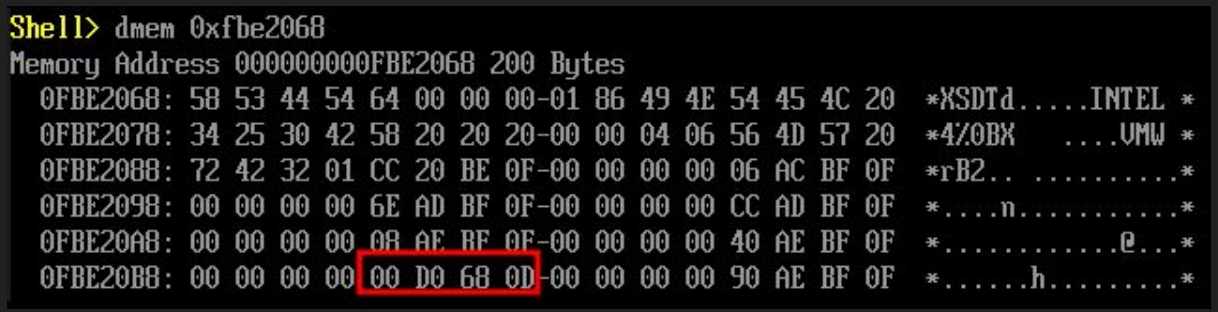
- This binary was used by “**Lenovo Service Engine**” (LSE) to smuggle a big blob of PE files, registry keys and config data from firmware into Windows.
- It uses **Lenovo LUFT table**, that isn’t part of ACPI spec, only Lenovo’s binary parses it.
- This lets an attacker install an arbitrary driver or service completely from ACPI, no disk write needed until Windows does it for you.



Lenovo LUFT table

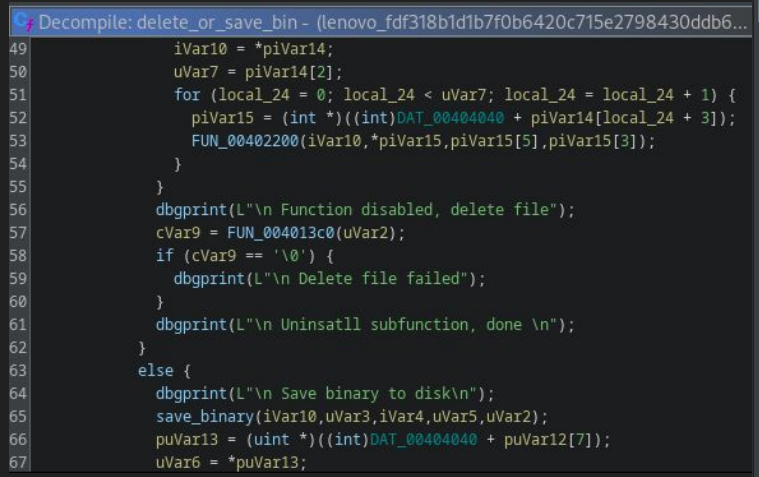
We construct a new LUFT table that:

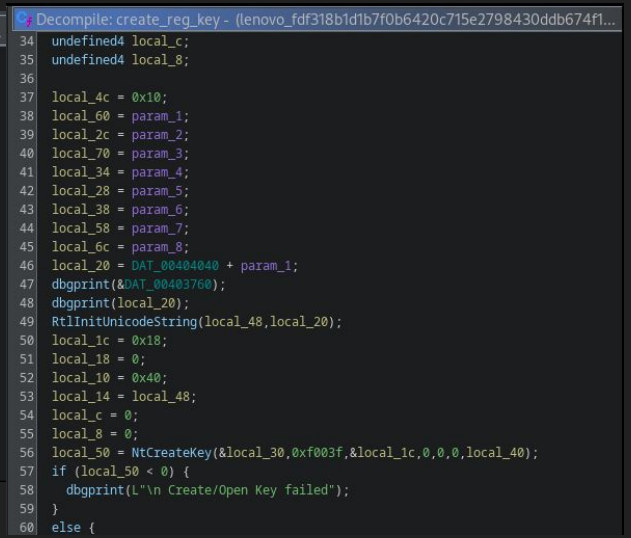
- **Drops hell.sys** (read from vulndriver.sys in the repo, this is a signed vulnerable driver).
- **Creates a service key** under HKLM\SYSTEM\CurrentControlSet\Services\2srvdriver.
- Sets DisplayName, ImagePath, Type, Start, ErrorControl values.
- **Re-use slot #6 in the XSDT** (originally the OEM's WSMT entry) to point at the LUFT buffer instead, and fix the XSDT checksum.



What does the lenovo native app do?

- **Looks for** custom ACPI table named **LUFT**.
- **Writes** the embedded **binary to disk** calling the function labeled as “save_binary” to C:\Windows\System32\Drivers\XXXX.sys
- Then **invokes create_reg_key()** on every sub-descriptor, which in turn writes or patches the registry so the binary will launch automatically.

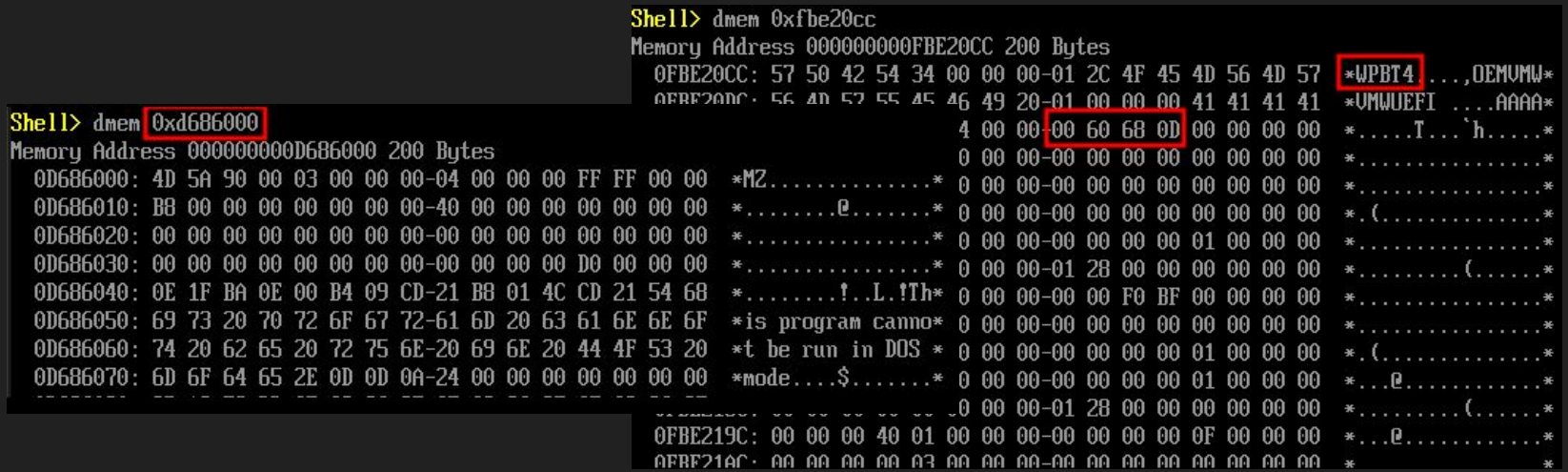




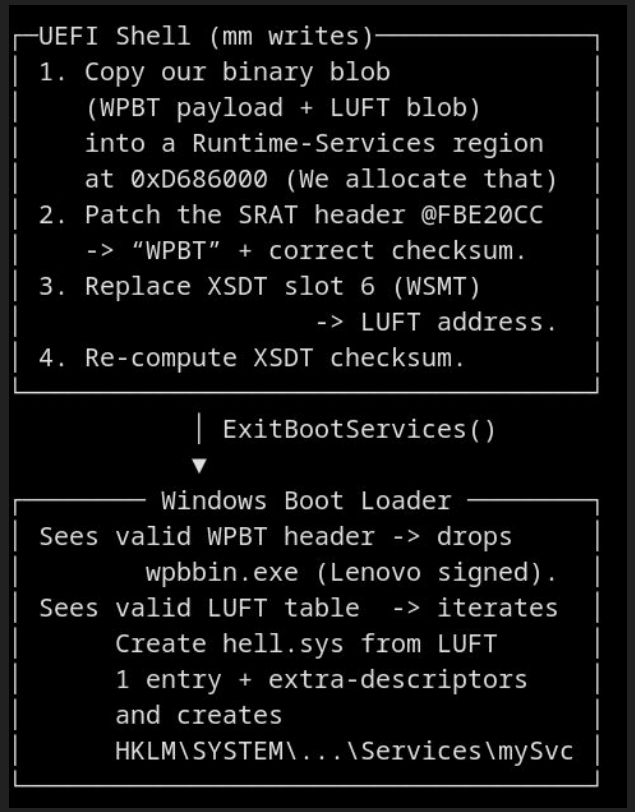
WPBT

To construct our WPBT:

- Find a **writable runtime-memory** space (1 MiB at BIN_ADDR, we allocated it before with our .EFI) and **copy our real payload** (the Lenovo-signed native app) there
- **Overwrite** the **first 16 bytes of the in-RAM SRAT** with that WPBT header



To summarize...



- Firmware finishes POST, enters **ExitBootServices()**; **RT-memory** still **contains our injected blobs**.
- Windows loader scans the XSDT, **find our LUFT**.
- Probes the RAM page that used to be **SRAT** but **now begins with "WPBT"**; sees a valid length & checksum, accepts it as the WPBT.
- **smss.exe** (session manager) copies the Lenovo-signed EXE from the WPBT to disk and **executes it as NtProcessStartup**.
- Lenovo **native signed app** (started later by RunServicesOnce) **parses the LUFT**; **drops hell.sys**, writes the registry values, and starts the service (or schedules it for next boot, depending on the flags).
- On the next boot, Windows sees a legitimate driver entry under Services\2srvdriver, **loads hell.sys** at kernel time, and the **system is compromised**.


Demo
Time!
meme-arsenal.ru

03

Vulnerable driver usage and exploitation

BYOVD – From EFI to Kernel without detection

- We use legitimate but vulnerable drivers to gain kernel-level access.
- These drivers expose powerful memory access primitives like:
- **ZwMapViewOfSection**, **MmMapIoSpace**, **MmCopyMemory**, **ZwTerminateProcess**, etc.
- Poorly implemented IOCTLs allow user-mode control over kernel functions.

Once loaded, we use them to:

- Bypass security mechanisms.
- Elevate privileges by modifying **EPROCESS** tokens (SYSTEM token steal).
- Hook and modify kernel structures.
- Disable callbacks: process/thread/image notifications via **PspCreateProcessNotifyRoutine**.
- These drivers are often found on Microsoft Update Catalog or via VirusTotal RetroHunt.
- Our custom EFI loaders plant the drivers silently during early boot.

What Does the Driver Do?

A signed Huawei kernel-mode driver that enforces process protection, blocks access to targeted processes, and relaunches them automatically.

- **HwAiGalleryGuardDriver.sys** is a signed driver from Huawei, typically installed as part of the HwAIServiceSetup package. It runs with kernel privileges and remains active even after the associated software is uninstalled.

- The driver registers a process creation and termination callback via **PsSetCreateProcessNotifyRoutine**, allowing it to monitor and react to specific processes defined by user-mode input.

- It also registers an object access filter using **ObRegisterCallbacks** to intercept PsProcessType object operations (like **OpenProcess**) and strip access rights such as **PROCESS_TERMINATE** from other processes trying to interfere.

- When a monitored process terminates, the driver automatically recreates it, using **APC injection** into a thread of a trusted process (services.exe), launching a hidden command like “sc.exe start myservice.exe”.

- This design provides a stealthy, persistent protection mechanism: the process is shielded from external control, and silently restored if killed all under kernel-level control and without any user-mode component being responsible.


```
NTSTATUS result; // eax
NTSTATUS ntStatus; // [rsp+20h] [rbp-18h]

sub_140001900("RegisterAutoResuming enter", Remove);
ntStatus = PsSetCreateProcessNotifyRoutine::NotifyRoutine, 0);
if ( ntStatus >= 0 )
{
  sub_140001900("RegisterAutoResuming sucess");
  LOBYTE(result) = 1;
}
else
{
  sub_140001900("RegisterAutoResuming Fail, status=0x%08X.", (unsigned int)ntStatus);
  LOBYTE(result) = 0;
}
return result;
```

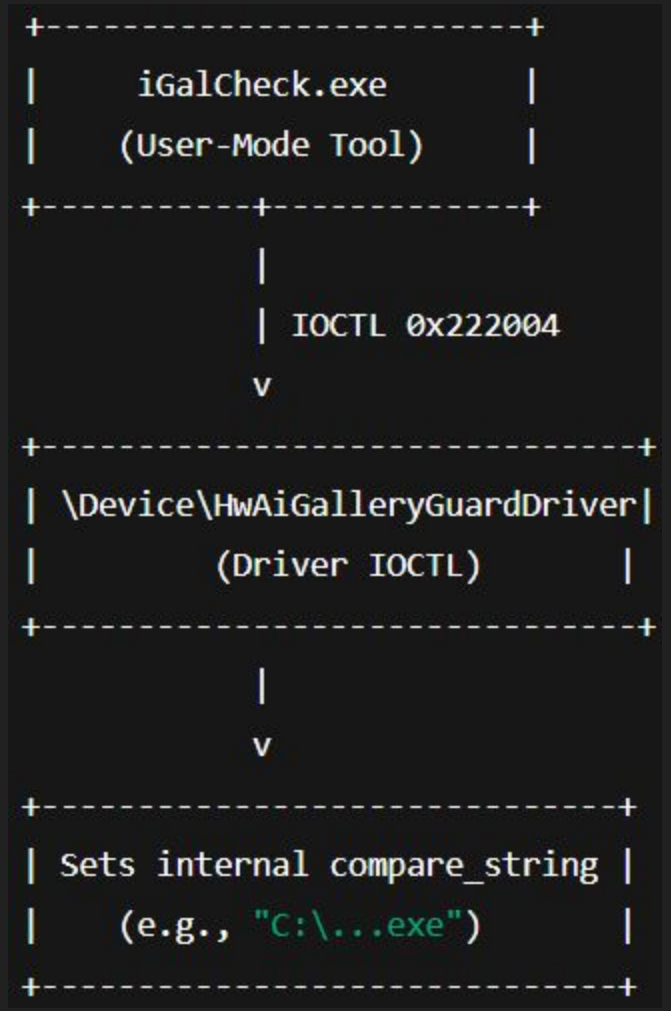
```
NTSTATUS status; // [rsp+40h] [rbp-18h]
void *threadHandle; // [rsp+48h] [rbp-10h] BYREF

threadHandle = 0i64;
status = PsCreateSystemThread(&threadHandle, 0x1FFFFFu, 0i64, 0i64, 0i64, (PKSTART_ROUTINE)StartRoutine, 0i64
if ( threadHandle )
  ZwClose(threadHandle);
if ( status < 0 )
  sub_140001900("CreateProcessByApc CreateThread Fail, status=0x%08X.", (unsigned int)status);
else
  sub_140001900("CreateProcessByApc CreateThread OK ");
```

```
v1 = ObRegisterCallbacks(&CallbackRegistration, &RegistrationHandle);
if ( v1 >= 0 )
{
  sub_140001900("RegisterForbidingKill sucess");
  return 1;
}
else
{
  sub_140001900("RegisterForbidingKill Fail, status=0x%08X.", (unsigned int)v1);
  return 0;
```

How does it interact with User-Mode?

The driver is fully controllable from user-mode without authentication or privilege checks.

- It exposes a device object (**\\.\HwAiGalleryGuardDriverControl**) that accepts IOCTL requests from any user-mode process.
- The included tool iGalCheck.exe communicates with the driver using commands like /protect and /unprotect.
- These commands trigger the **IOCTL 0x222004**, which does not validate input or require administrative privileges.
- The buffer passed contains a Unicode string with the path to the executable to protect, typically like "C:\Windows\System32\notepad.exe".
- No input sanitization, signature checking, or access control is performed any process can be protected arbitrarily.




```
cookie = -2i64;
deviceHandle = (void *)deviceContext[10];
if ( !deviceHandle )
  return 0;
bytesReturned = 0;
if ( !DeviceIoControl(deviceHandle, 0x222004u, inBuffer, 0x194u, outBuffer, 0x194u, &bytesReturned, 0i64) )
{
  GetLastError();
  errorMsgStrObj = (void *)sub_140005160(tempStrObjC);
  errorStringPtr = sub_1400071E0(errorMsgStrObj);
  tempStrMetaA = 0i64;
  *(_OWORD *)Src = *(_OWORD *)errorStringPtr;
  tempStrMetaA = *(__m128i *)(errorStringPtr + 16);
  *(_QWORD *)(errorStringPtr + 16) = 0i64;
```

```
unprotectCheckView = &svcStartTable;
if ( cmdBufSize >= 0x10 )
  unprotectCheckView = (SERVICE_TABLE_ENTRYW *)serviceNameRaw;
if ( cmdLen != 9 || memcmp(unprotectCheckView, "unprotect", 9ui64) )
{
  qProtectCheckView = &svcStartTable;
  if ( cmdBufSize >= 0x10 )
    qProtectCheckView = (SERVICE_TABLE_ENTRYW *)serviceNameRaw;
  if ( cmdLen == 8 && !memcmp(qProtectCheckView, "?protect", 8ui64) )
  {
    sub_140001280();
  }
  else
  {
    sub_140001010("Parameters: \"-\" or \"/\" in front\n", qLoadCheckView);
    sub_140001010("  enable | disable | inatall | uninstall | ?install | load | unload | ?load | protect "
                  "| unprotect | ?protect \n");
  }
```

What internal validation logic does it use?

The driver's validation is minimal and based solely on an exact path string comparison.

- Upon receiving a request, the driver retrieves the target process's PEB using **PsGetProcessPeb** and KeStackAttachProcess.
- It extracts the process's CommandLine and compares it with an internal compare_string (set earlier **via IOCTL**).
- The comparison uses **RtlCompareUnicodeString**, which is case-sensitive and does not normalize path syntax.
- No signature, hash, or executable metadata is verified only a raw string match on the command line.

How does it maintain persistence?

It uses driver-level callbacks and APC injection to ensure process resurrection.

- Even after uninstalling the software, the driver remains loaded, as the uninstall routine does not remove it.
- Manual removal is required using tools like sc delete, devcon remove, and sc stop for associated services.
- When a protected process exits, the driver launches a new process using APC injection into a thread of services.exe.

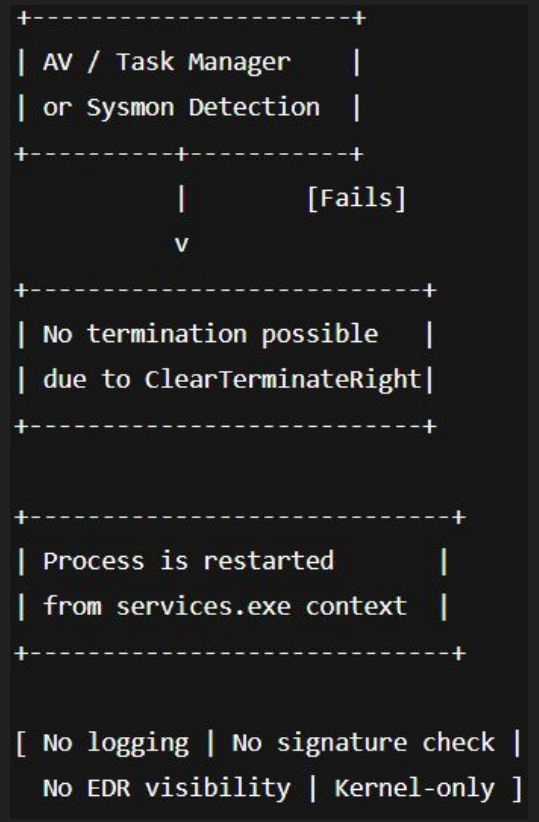

```
NTSTATUS status; // [rsp+40h] [rbp-18h]
void *threadHandle; // [rsp+48h] [rbp-10h] BYREF

threadHandle = 0i64;
status = PsCreateSystemThread(&threadHandle, 0x1FFFFFu, 0i64, 0i64, 0i64, (PKSTART_ROUTINE)StartRoutine, 0i64
if ( threadHandle )
  ZwClose(threadHandle);
if ( status < 0 )
  sub_140001900("CreateProcessByApc CreateThread Fail, status=0x%08X.", (unsigned int)status);
else
  sub_140001900("CreateProcessByApc CreateThread OK ");
```

- The routine CreateProcessByApc constructs a command line like sc.exe start hell and injects it via KeInsertQueueApc.
- This kernel-level process creation bypasses standard security hooks, allowing silent and persistent restarts.

What Security Mechanisms Does It Ignore?

It bypasses typical user-mode protections and disables process control.

- Removes the ability to terminate protected processes by clearing **PROCESS_TERMINATE** rights (**ClearTerminateRight**).
- Evades user-mode monitoring tools (like Task Manager or Sysmon) by performing operations entirely in kernel space.
- Uses legitimate system processes (e.g., services.exe) as injection targets to hide malicious activity.
- No signature validation means even untrusted or tampered binaries can be protected.
- These characteristics make the driver a powerful primitive for “rootkits”, EDR evasion, and stealth persistence.




DEMO TIME!
38
... WHAT COULD POSSIBLY GO WRONG
makeameme.org

Conclusions

- Secure Boot isn't a silver bullet
  - Signed binaries exploitable if DBX not updated.
- UEFI: overlooked & powerful attack surface
  - Early execution bypasses traditional EDR.
- EFI loaders chain silently into custom payloads
  - Legitimate ESP entries provide persistent footholds.
- BYOVD (Bring Your Own Vulnerable Driver)
  - Abuses legitimate drivers for kernel implants.
- Exploits poor IOCTLs for kernel R/W & privilege escalation.
  - Disable critical protections
- All components are signed, trusted & load silently
  - Publicly sourced drivers (Microsoft Catalog, VT RetroHunt).
- Full persistence across reboots
  - Effective even with BitLocker/VBS enabled, with tweaks ;)

References

- ESET. (s. f.). Machine Learning and UEFI. https://web-assets.esetstatic.com/wls/en/papers/white-papers/ESET_Machine_Learning_UEFI.pdf
- HackingThings. (s. f.). SignedUEFIShell [GitHub repository]. GitHub. https://github.com/HackingThings/SignedUEFIShell/tree/main
- SOC Investigation. (2023). UEFI persistence via wpbbin: Detection & response. https://www.socinvestigation.com/uefi-persistence-via-wpbbin-detection-response/
- Sophos. (2023, junio 2). Researchers claim Windows backdoor affects hundreds of Gigabyte motherboards. https://news.sophos.com/en-us/2023/06/02/researchers-claim-windows-backdoor-affects-hundreds-of-gigabyte-motherboards/
- tandasat. (s. f.). WPBT-Builder [GitHub repository]. GitHub. https://github.com/tandasat/WPBT-Builder?tab=readme-ov-file
- Persistence Info. (s. f.). WPBBin. https://persistence-info.github.io/Data/wpbbin.html
- Unified Extensible Firmware Interface Forum. (s. f.). UEFI Revocation List File. https://uefi.org/revocationlistfile
- Microsoft. (s. f.). secureboot_objects [GitHub repository]. GitHub. https://github.com/microsoft/secureboot_objects
- HackingThings. (s. f.). OneBootloaderToLoadThemAll [GitHub repository]. GitHub. https://github.com/HackingThings/OneBootloaderToLoadThemAll/
- Knopper, K. (s. f.). Knoppix and UEFI. https://www.knopper.net/knoppix/knoppix-uefi-en.html
- br-sn. (n.d.). Removing Kernel Callbacks Using Signed Drivers. Retrieved from https://br-sn.github.io/Removing-Kernel-Callbacks-Using-Signed-Drivers/
- br-sn. (n.d.). CheekyBlinder [GitHub repository]. GitHub. Retrieved from https://github.com/br-sn/CheekyBlinder
- VL. (2021). Removing Process Creation Kernel Callbacks. Medium. Retrieved from https://medium.com/@VL1729_JustAT3ch/removing-process-creation-kernel-callbacks-c5636f5c849f
- lawiet47. (n.d.). STFUEDR [GitHub repository]. GitHub. Retrieved from https://github.com/lawiet47/STFUEDR
- hfiref0x. (n.d.). KDU (Kernel Driver Utility) [GitHub repository]. GitHub. Retrieved from https://github.com/hfiref0x/KDU
- TheCruZ. (n.d.). kdmapper [GitHub repository]. GitHub. Retrieved from https://github.com/TheCruZ/kdmapper
- Sophos. (2022, October 4). BlackByte ransomware returns, abuses RTCore64.sys driver to disable kernel callbacks. Sophos News. Retrieved from https://news.sophos.com/en-us/2022/10/04/blackbyte-ransomware-returns/

https://github.com/0xedh/DEFCON33-KillChainReloaded

Special mention to @sanguinawer for helping us out!!!!

Thanks to @s4dbrd and JC